Deep Residual Learning for Image Recognition 논문 리뷰

1. Introduction

💡 논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

층을 깊게 만들면? → 성능 향상 but... 그렇지 않은 경우가 있음

언급되지 않은 함수를 학습하는 대신, 입력을 참고해서 기존에 언급되었던 함수를 참조해 층을 재구성한다.

“residual network”

Deep CNN은 이미지 분류 분야에 혁신을 가져오고 Deep 네트워크는 저-고수준의 특징과 분류기를 통합함

⇒ 최근 연구는 네트워크의 깊이는 매우 중요하단 걸 시사했다.

Depth의 중요성에 기인하여, 질문 하나가 떠오른다.

더 나은 신경망을 학습하는 것이 단순히 레이어를 더 많이 쌓는 것만큼 쉬운 일일까요?

해당 task에서 기존 연구 한계점

Overfitting Problem

Neural Networks는 층이 깊이질수록 성능이 좋아질 거란 가능성이 있지만, 실제로는 학습이 점점 어려워지는 문제가 발생한다.

1. Gradient Vanishing
2. Gradient Exploding

⇒ 중간 정규화 계층과 SGD로 해결

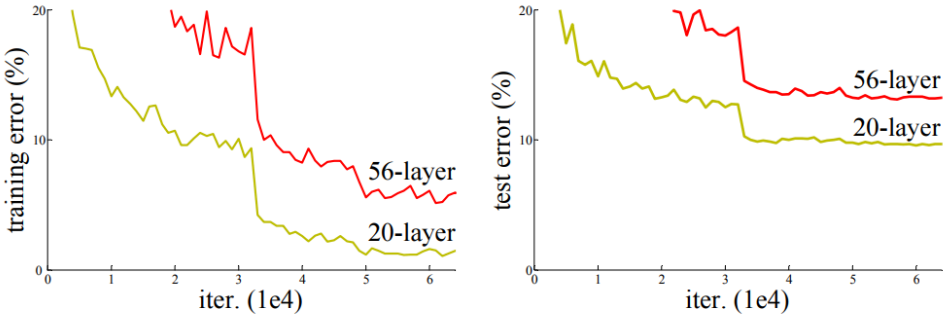


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

과적합 문제X, 적절한 깊이의 모델에 더 층을 쌓을 때 오히려 성능이 급격하게 저하되는 문제 → 훈련 오류 증가

overfitting 문제가 아닌 optimization문제

이 문제가 바로 **Degradation problem** 이다!

논문의 contributions

Deep Residual Learning framework 도입

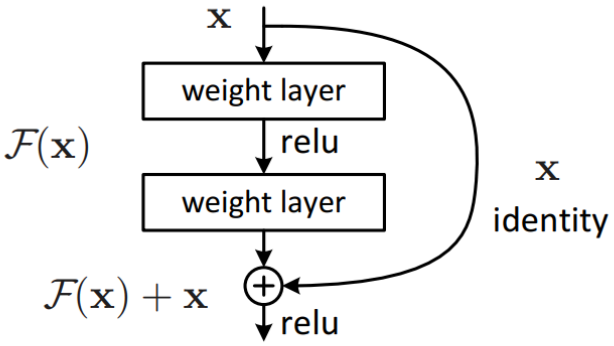


Figure 2. Residual learning: a building block.

기존

- layer들이 전체 함수(mapping)을 학습: H(x)

제안

- layer들이 Residual(차이)를 학습: H(x)=F(x)+x

  → Residual 함수를 학습하는 게 더 쉬움 (ex. identity mapping)

Shortcut Connection: block의 입출력을 서로 더해줌

= skip connection: 입력을 그래도 전달

→ 추가 파라미터, 연산량 증가 X

2. Related Work

💡 Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
제안 방법의 차별성을 어떻게 표현하고 있는가

Residual Representations

VLAD, Fisher Vector : Residual vector 활용하는 기법

→ 매우 강력한 이미지 특징 표현 방법

Shortcut Connections

- MLP: 입출력 연결하는 linear layer 추가
- vanishing gradient: 기울기 소멸 문제 해결을 위해 직접 연결 중간층 추가
- Inception layer: shortcut branch + deep branch 구조
- Highway Network
  - shortcut connection + **gate** functions
  - gate는 shortcut 사용 여부를 결정
  - 논문 속 모델과의 차이 Residual을 항상 가짐, shortcut 항상 작동

3. 제안 방법론: Deep Residual Learning

💡 Introduction에서 언급된 내용과 동일하게 작성되어 있는가
Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

Main Idea:

Residual Learning

기존:

입력 x에 연산을 수행하여 output을 얻음

Residual Learning:

출력을 F(x)+x 로 재구성하여 입력 부분을 출력으로 그대로 넘겨줌

Identity Mapping by Shortcuts

Building bloke defined as:

- F(x, {W_i})는 residual mapping 식을 의미함.

(1) x와 F(x)의 차원이 같은 경우

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \tag{1}$$

(2) 차원이 다른 경우

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}. \tag{2}$$

- linear projection W_s 를 적용해 차원을 맞춰준다

Network Architectures

본 논문에서는 plain과 residual net을 실험했다.

Plain Network

- VGG net에서 영감받음
- 합성곱층 3x3 filter
- 규칙
  - 동일한 feature map size → 같은 수의 필터를 가져야 함.
  - feature map size와 filter 수는 반 비례

Residual Network

- plain network를 기반으로 shortcut connections을 삽입하여 counterpart residual version으로 바꾼 것
- shortcut은 직접적으로 input-output
- 차원 수가 많아질수록
  - shortcut이 identity mapping을 수행
  - projection shortcut이 수행

  → 둘 다 stide=2의 다운샘플링을 수행함

Contribution

- **Resudual block** 구조
- **Skip-Connection** 도입

4. 실험 및 결과

💡 Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

Dataset

ImageNet

- ImageNet 2012 classification dataset
- 1000개의 class

CIFAR-10

- 50k의 train data와 10k의 test image로 구성
- 10개의 class

Baseline

- **VGG**
- **Plain CNN**

Plain Netwroks

- 34-layers plain network가 가장 높은 valid error를 보임
- 기울기 소멸 문제가 아닌 degradation problem이 발생한 것으로 보임

→ BN 사용 했기 때문: forward propageted signal을 non-zero로 보장

Residual Networks

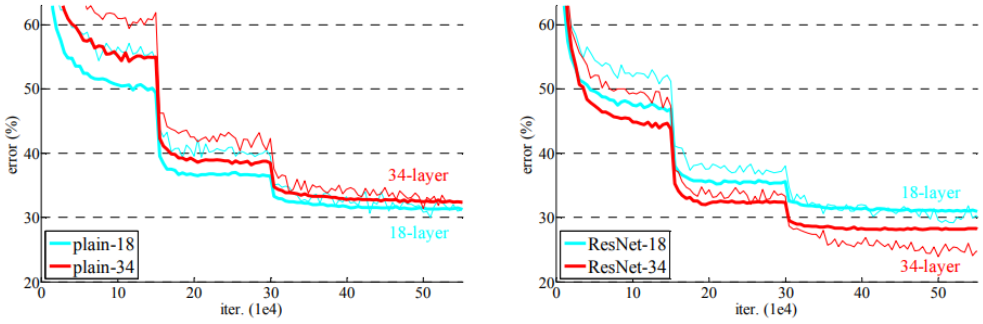
- plain net에 3x3 필터의 **shortcut**을 추가함
- identity mapping & zero-padding 추가

  → 추가 파라미터 X

결과

(1) ImageNet 결과

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	**25.03**



기존 네트워크는 깊이가 깊어질수록 error도 커지지만, ResNet모델의 경우 오히려 성능이 개선됨을 알 수 있다.

Plain Netwroks

- 34-layers plain network가 가장 높은 valid error를 보임
- 기울기 소멸 문제가 아닌 degradation problem이 발생한 것으로 보임

→ BN 사용 했기 때문: forward propageted signal을 non-zero로 보장

Residual Networks

- 34-layers가 18-layers 보다 오류 낮음
- 34-layer ResNet이 낮은 학습 오류와 일반화가 잘됨을 보임

  → degradation problem이 잘 해결됨

  → 층을 깊게해도 성능이 향상됨

Identity vs Projection Shorcuts

(A) zero-padding shortcuts & all shortcuts params free

(B) projection shortcut & other shortcuts are identity

(C) all shortcuts are projections

model	top-1 err.	top-5 err.
VGG-16 [40]	28.07	9.33
GoogLeNet [43]	-	9.15
PReLU-net [12]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	**21.43**	**5.71**

B>A

- A의 zero-padded된 차원이 residual learning을 가지지 않음

C>B

- projection shortcuts로 인해 도입된 추가 params

** 하지만 A/B/C 간의 성능 차이가 매우 작기 때문에 projection shortcuts은 degradation 문제 해결에 큰 영향을 미치지 않는다.

Deeper Bottleneck Architectures

train time 절약을 위해 bottleneck 디자인으로 블록을 설계

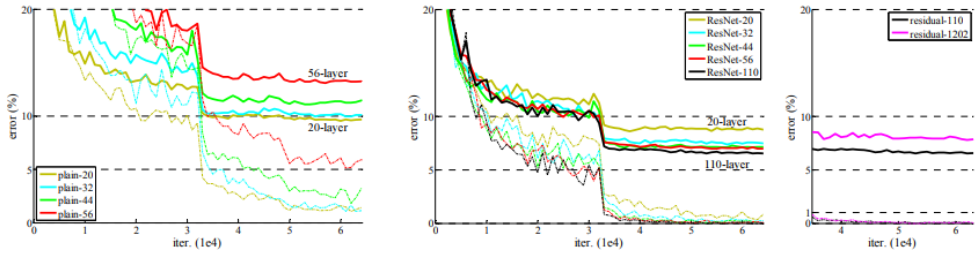
- residual function F에서 3개의 layer stack 사용

- 각각 1x1, 3×3, 1×1 conv 합성곱층
  - 1×1 : restoring dimensions
  - 3×3: bottleneck층, 더 작은 input/output dimension

method	top-1 err.	top-5 err.
VGG [40] (ILSVRC'14)	-	8.43†
GoogLeNet [43] (ILSVRC'14)	-	7.89
VGG [40] (v5)	24.4	7.1
PReLU-net [12]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	**19.38**	**4.49**

→ 152 layer까지도 34-layer 보다 더 정확함을 결과상으로 알 수 있다.

(2) CIFAR-10 결과 및 분석



- 왼쪽과 중간 그래프를 보면 110 layer까지 쌓은 경우에는 오히려 네트워크의 성능이 향상된 것을 알 수 있다.
- 하지만 오른쪽 그래프에서 층을 1202개로 극단적으로 깊이 만든 경우에는 오히려 error가 증가하여 성능이 하락하였음을 알 수 있다.

  → 다만 training error의 경우 성능이 향상됐기에 논문에서 주로 다루는 degradation problem이 아닌 overfitting problem이 발생했음을 알 수 있다.

5. 결론 (배운점)

💡 layer를 극단적으로 올린 경우에 성능이 하락하였는데, 이것이 overfit인지 degradation 문제인지 더 상세하게 실험하고 조사하는 과정이 생략된 부분이 아쉬워서 추가로 알아보고 싶은 생각이 들었다.
입력을 추가로 아웃풋으로 그대로 넘긴다는 아이디어는 매우 간단한데 이를 통해 degradation 문제가 해결된 것을 보고 간단한 아이디어로 모델 성능 개선을 할 수 있다는 부분이 참신했다.